CHAPTER

THREE

C PROGRAMMING

3.1 The first program

The classic "Hello World!" in C looks like this:

```
/*
  Hello World

  Demonstration showing a simple program that is self-contained.

  Program written by David Chopp

  Editing History:

  2/14/14: Initial draft
  7/29/17: Changed formatting of comments
*/

// Programs often require functions from built-in libraries,
//    or user-written files
// Interface information for those functions are stored in header files
//    and loaded before the code is compiled.
// This program uses terminal I/O, so it uses the stdio library.
//    This is very common...

#include <stdio.h>

/*
  int main(int argc, char* argv[])

  The main program is the starting point for a C program.
  There can only be one "main" function.  All others must be
  uniquely named something else.

```

```
30   Inputs:
31   int argc: When the program is run, argc contains the number of
32   arguments given to the program, e.g. if you type
33   "hello there" to the command line, then argc will have
34   the value 2.
35
36   char* argv[]: When the program is run, argv contains an array of
37   strings, where each string is what was typed, e.g. if
38   you type "hello there" to the command line, then
39   argv[0] will have the value "hello" and argv[1] will
40   have the value "there"
41
42   Outputs:
43   int: The main program should return 0 if successful, and return
44   a non-zero value if it encountered an error.
45 */
46
47 int main(int argc, char* argv[])
48 {
49   // It is important to get in the habit of thoroughly commenting
50   // your code.
51
52   printf("Hello World!\n");
53
54   // All done, no errors, so return 0
55
56   return 0;
57 }
```

Here we review several features of this code. Lines 1-12 are comments and any line beginning with `//` also indicates that it is a **comment**. A comment is used towards readability and documentation. Compiler ignores everything that is commented. Places to add comments are the top of each file, to include the information about the contents of the file, at the top of each function, to describe the function, the arguments, and the output of the function, and descriptive information about the logical steps of the program.

In line 47, `main` is a special name that indicates the place in the program where execution starts. At the beginning, the program executes the first statement in `main` and it continues, in order, until it gets to the last statement, and then it exits. All statements should end with a semicolon (`;`).

C uses squiggly-braces ({ and}) to group things together. In this case, the `printf` statement is enclosed in squiggly-braces, indicating the *scope* of `main`. Also, notice that the statement is indented, which helps to show visually which lines are inside the definition. We showed earlier how to compile and run this program. The phrases that appear in quotation marks are called **strings** which can contain any

combination of letters, numbers, punctuation marks, and other special characters. The the backslash in `printf` \ is to add a new line to the output.

Unlike most languages, C has no input or output statements. Instead, a set of I/O functions is provided as a part of the standard library. One such function is `printf`. Prior to calling a standard library function, we should inform to compiler where to find the function. In the case of standard I/O, including calling `printf`, we do this by using `#include <stdio.h>`. This is called a directive for the C *preprocessor*, which processes the source code before compilation. In this case, the header file is `stdio.h`. The header file must be placed at the top of the source file, prior to any function definition. We can think of the header file being replaced by its source file that

3.2 Variables

C is a powerful language in terms of its ability to manipulate **variables**. There are different types of variables. When variables are declared in C, their types need to be determined. Common arithmetic data types are:

Type name	Size (byte)	Description
`char`	1	Single Character
`bool`	1	Boolean
`int`	4	Integer
`float`	4	Single precision floating point
`double`	8	Double precision float
`short int`	2	Short integer
`long int`	8	long integer
`long double`	16	Quadruple precision floating point
`unsigned char`	1	Unsigned single character
`unsigned int`	4	Unsigned integer
`unsigned short int`	2	Unsigned short integer
`unsigned long int`	8	Unsigned long integer

While the sizes listed above are typical sizes, we must use `sizeof` function to determine the exact size values on a particular system. A declaration of a variable looks like this:

```
char foo;
```

to create a `char` variable. Obviously, the range and precision of these types are important to a programmer. For example:

Type name	Limits Macros	Range
`char`	[`CHAR_MIN, CHAR_MAX`]	[-128, 127]
`int`	[`INT_MIN, INT_MAX`]	[-2147483648, 2147483647]
`float`	[`FLT_MIN, FLT_MAX`]	$[1.175494 \times 10^{-38}, 3.402823 \times 10^{38}]$
`double`	[`DBL_MIN, DBL_MAX`]	$[2.225074 \times 10^{-308}, 1.797693 \times 10^{308}]$

Macros are character strings that replace their defined value at the compilation time. For example, in `limits.h`, `INT_MAX` is defined as

```
#define INT_MAX 2147483647;
```

Another important value that a programmer needs to know is the machine epsilon, an upper bound on the relative approximation error due to rounding in floating point arithmetic. For example, for a `float` type, the machine precision is 1.192093×10^{-7}, meaning that the single precision float can accurately represent a number of 7 significant digits. There is a speed trade-off in choosing a greater precision. Higher precision arithmetic is slower. The macro for machine epsilon is defined, for example for float, as `FLT_EPSILON`.

Variables are assigned values, either when they are declared or later, before being used. This is done with an **assignment statement**.

```
int index;
long double x = 11.111;
index = 0;
```

We should also note that uninitialized variable are not set to zero by default. Also, for example, a string cannot be stored in an `int` variable; in general, variables and values have the same type.

Generally, any variable names can be used; however, certain keywords are reserved in C because they are used by the compiler to parse the structure of the program. For example, name of a variable cannot be `int`, `char`, `void`, and so on. The complete list of keywords can be found at `http://www.ansi.org/`.

3.3 Inputting and Outputting

The input and output are mainly used in two ways: outputting diagnostic information, storing data. We will review a program that reads and writes files, parses input, as well as terminal I/O.

We can output the value of a variable using `printf` command:

```
int hour = 11, minute = 59;
```

```
char colon = ':';

printf ("The current time is: \n");
printf ("%d%s%d\n", hour, colon, minute);
```

This program creates two integer variables named `hour` and `minute`, and a character variable named `colon`. It assigns appropriate values to each of the variables and then uses `printf` to output statements to generate the following:

```
The current time is:
11:59
```

"\n" (called a escape sequence) represents a new line. The variables `hour` and `minute` are of `int` type and `%d` tells `printf` that it corresponding argument has an integer type, when displaying its value. In general, `printf` appears as:

```
int printf(char *format, argument1, argument2, ... )
```

`scanf()` is similar to `printf`, but it reads input and appears as:

```
int scanf(char *format, addr of argument1, addr of argument2, ... )
```

The first argument will be a format string, which specifies the expected form of the input. Below is the list of format specifiers, for both `scanf` and `printf` :

Format specifier	Description	Supported data types
`%c`	Character	`char`
`%d`	Signed integer	`int`
`%e`	Scientific notation of float values	`float, double`
`%f`	Floating point	`float`
`%g`	Similar as %e	`float, double`
`%hi`	Signed integer (short)	`short int`
`%hu`	Unsigned integer (short)	`unsigned short int`
`%ld`	Long integer	`long int`
`%lf`	Double precision floating point	`double`
`%Lf`	Quadruple precision floating point	`long double`
`%o`	Octal number	`int`
`%s`	String	`*char`
`%u`	Unsigned integer	`unsigned int`
`%x`	Hexadecimal of unsigned integer	`int`
`%n`	Prints nothing	
`%%`	Prints %	

The second argument in `scanf` must be the memory address, not the value, unlike `printf`. Reading and writing files are important when dealing with large amount

of data. Let's look at one example:

```c
#include <stdio.h>
#include <math.h>

/*
  int main(int argc, char* argv[])

  The program opens a new file with the given name, writes some data to
  it, then reads those numbers back in and prints them to the screen.

  Inputs: argc should be 2,
  argv[1] will contain the filename to be created

  Outputs: creates the file and writes some data to it.
*/

int main(int argc, char* argv[])
{
  // First thing is to create the file we will use for storage
  // the "w" indicates we are opening the file for writing
  FILE *fileid = fopen(argv[1], "w");

  // Once the file is open, we can write to it with fprintf functions
  //      that act the same as the printf commands we learned earlier
  // Note the use of M_PI. This is a useful macro that defines the
  //      number pi and is found in the header file math.h
  fprintf( fileid, "%d %6.3f %s", 17, M_PI, "Hello World!\n");

  // When we're done writing, we must close the file so that it is
  //      terminated properly
  fclose(fileid);

  // To read the data back in, we must define the variables
  int num;
  double pi;
  char greeting[256];

  // The file is closed, so let's open it again, but this time
  //      use "r" to indicate that we will read the file
  // Note that we don't have to redeclare fileid because
  //      we did it already
  fileid = fopen(argv[1], "r");

  // Now we use fscanf the same way we learned scanf
  // Note that greeting does not have an ampersand because greeting is
  //      an array, so greeting has type char* already
  fscanf( fileid, "%d %lf %s", &num, &pi, greeting);
```

```
47
48    // All done reading, so close the file
49    fclose(fileid);
50
51    // Now print the results.  Did we get what we expected?
52    printf("%d %lf %s\n", num, pi, greeting);
53    return 0;
54  }
```

The I/O is through the form of a `FILE*` type. Here we open the file with `fopen()`, it returns the `FILE*` (or `NULL` if failed) - "`w`" here means open a text stream for writing. All streams are automatically closed when the program exits, but for a good practice, it's better to explicitly close any files opened for reading or writing. `fprintf` and `fscanf` are similar to `printf` and `scanf` with an additional argument to stream data through reading or writing from a file. There is a macro defined as `EOF`. This is what `fscanf` will return when the end of the file has been reached. Here is another example of reading formatted text from a file `colorcode.txt`:

```
black 1
blue 2
white 3
gray 4
```

```
#include <stdio.h>

int main(void)
{
  FILE *fp;
  char color[1024];
  int code;

  fp = fopen("colorcode.txt", "r");

  while (fscanf(fp, "%s %f %d", color, &code) != EOF)
    printf("%s color code is: %d\n", color, code);
  fclose(fp);
}
```

The `fscanf()` function skips leading whitespace when reading, and returns `EOF` on end-of-file or error.

Reading and writing in text format is extremely inefficient; binary formats offer advantages in that they are more efficient, With binary files, a raw stream of bytes are processed or reading and writing. Below is an example

```
#include <stdio.h>

int main(void)
{
  FILE *fp;
  unsigned char bytes[6] = {5, 37, 0, 88, 255, 12};

  fp = fopen("output.bin", "wb"); // wb mode for "write binary"!

// fwrite arguments are:
// * Pointer to data to write
// * Size of each "item" of data
// * Count of each "item" of data
// * FILE*

  fwrite(bytes, sizeof(char), 6, fp);

  fclose(fp);

  fp = fopen("output.bin", "rb"); // rb for "read binary"!

  while (fread(&c, sizeof(char), 1, fp) != 0)
    printf("%d\n", c);
}
```

The arguments of `fwrite()` are the size of each item how many items to be read. `fread()` here reads back the data from the file, which is in binary format.

3.4 Operations

Basic arithmetic **Operators** are `+, -, *, /, (, )` with mathematical operations order of precedence. Recall also that processors have special hardware just for performing floating-point operations. Integer division may result in an unexpected output. When both of the operands of an integer division are integers, the result is also supposed to be an integer, always rounding down by definition, regardless. For example,

```
  int x = 1;
  int y = 2;
  int z = x/y; \\ z will be assigned 0
  int r = (x + 2)%y; \\ r will be assigned 1
```

A type cast can be specified, for example, as follows:

```
int x = 1;
int y = 2;
double w = x / y; \\ w will be assigned 0.0
double z = (double)x/y; \\ z will be assigned 0.5
```

Below is an example of using increment and decrement shortcuts:

```
#include <stdio.h>

int main(int argc, char* argv[])
{
  // We'll use two integer variables: val and i
  // val will show the result of the equation, and
  //      i will be the variable being incremented or decremented
  int val = 0;
  int i = 1;

  // Before/after each operation, you can see the resulting values of
  //      the increment and decrement operators
  printf("Initial values:\nval = %d, i = %d\n\n", val, i);
  val =  ++i;
  printf("After val =  ++i:\nval = %d, i = %d\n\n", val, i);
  val = i++;
  printf("After val = i++:\nval = %d, i = %d\n\n", val, i);
  val = --i;
  printf("After val = --i:\nval = %d, i = %d\n\n", val, i);
  val = i--;
  printf("After val = i--:\nval = %d, i = %d\n\n", val, i);


  return 0;
}
```

The same mathematical operations that work on integers also work on characters:

```
char letter;
letter = 'a' + 1;
printf("%s", letter);
```

outputs the letter `b`.

3.5 Boolean Algebra

There is another type in C called **boolean**, taking only values of `true` and `false`. The condition inside an `if`statement or a `while` statement is a boolean expression.

Also, the result of a comparison operator is a boolean value. For example:

```
int x;
...

if (x) {
  \\ do something
}
else {\\ do somthing else
}
```

where `if (x)` is the same as `if (x != 0)`. The basic boolean operations include the operator `==` compares two integers and produces a boolean value and `!=` which produces true if the values of operands are not equal. The values `true` and `false` are keywords in C. For example,

```
while (true) {
  // loop forever
}
```

As mentioned earlier, boolean type is called **bool**.

```
bool YES;
YES = true;
bool NO = false;
```

The result of a comparison operator is a boolean, for example:

```
bool even = (n%2 == 0);      // true if n is even
bool positive = (x > 0);     // true if x is positive
```

and then use it as part of a conditional statement later

```
if (even) {
  printf("n is even\n");
}
```

The logical operators are: `AND` , `OR` and `NOT`, given by the operators: `&&`, `||` and `!`. For example:

- `x > 0 && x <= 10` true only if `x` is greater than zero AND less than or equal 10.
- `(x != 10)` is false if `x` is not equal 10.
- `x <= 0 || x > 10` true if either if `x` $\leq$ `0` is true OR the `x > 10`.
- `max = x > y ? x : y` assigns the larger of two values `x` and `y` to `max`

Functions can also return `bool` values. For example:

```
bool isPositive (int x)
{
  if (x >= 0) {
    return true;
  } else {
    return false;
  }
}
```

or

```
bool isPositive (int x)
{
  return (x >= 0);
}
```

Finally, it is important to note to never trust equality (or inequality) operations when dealing with floating point values.

3.6 Mathematical Functions

There are mathematical functions available to use in C. These functions are declared in `math.h`, which provides a set of built-in functions that includes most of the mathematical operations. The math functions are invoked similarly to their mathematical notation:

```
double x = abs(x); \\ assigns |x| to x
double y = sin (x); \\the same as sin(x)
```

Other examples are `log(x)` ($\ln(x)$), `log10(x)` (takes logarithms base 10), and `log2(x)` (takes logarithms base 2). The values used with `sin` and the other trigonometric functions (`cos`, `tan`) are in *radians*. π to 15 digits can be calculate it using the `acos` function. The arccosine (or inverse cosine) of -1 is π:

```
double pi = acos(-1.0);
```

3.7 Functions

Functions are the backbone of C. Functions provide ease of understanding of the code and help speed the programming development, by providing reuseability.

Complex algorithms can be broken up into small modules that are easy to understand, debug, and test. Functions simplify algorithms and if developed properly, for example being as generic as possible, the can be used as building blocks, mainly for later code developments.

We have already used the `main` function, where the execution of the program starts, but the syntax for `main` is the same as for any other function definition. Other function examples are `printf` and `scanf`. Functions take arguments (or no argument) and return a value (or nothing). The arguments and return value types are predeclared. Function definition look like:

```
  double NAME ( LIST OF PARAMETERS ) {
    STATEMENTS
}
```

The list of parameters is the data, or no data, passed to the function. In this case, the function returns a double precision value. A function declaration is included in the header file and appears like:

```
  double NAME ( LIST AND TYPE OF PARAMETERS );
```

Compiler need to know about the function declaration before it is used. It is useful to plan properly based on whether the function is used only in one file, or being called in many other files. Here are two usage examples of the same function:

```
#include <stdio.h>
#include <stdlib.h> // need this include for the atoi function

/* unsigned int squareOf(int n)

   returns the square of the input integer

   Input: number to be squared

   Output: square of input value
*/
// Note that squareOf is defined before it is called in main
unsigned int squareOf(int n)
{
  return n*n;
}


/*
  int main(int argc, char* argv[])

  Function prints the square of the integer argument

```

```
  Inputs: argc should be 2
  argv[1] will contain the integer input, can be positive or negative

  Outputs: Prints the square of the input
*/
int main(int argc, char* argv[])
{
  int n = atoi(argv[1]); // Get the input number

  printf("%d^2 = %u\n", n, squareOf(n));

  return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

//Function declaration
unsigned int squareOf(int n);

int main(int argc, char* argv[])
{
  int n = atoi(argv[1]); // Get the input number

  printf("%d^2 = %u\n", n, squareOf(n));

  return 0;
}

// Function defintion
unsigned int squareOf(int n)
{
    return n*n;
}
```

Execution always starts at `main`, and then statements are executed one at a time, in order; when reaching a function call, the execution goes to the function, executing all the statements there, and then returning to the line after the function call. Therefore there is a flow of execution, that needs to be followed. In the second example above, the codes can be put into separate files, namely the header file, where the function declarations are, a file that has the function definition, and a file that has the main function. You must use `#include` to insert the named file into the file that is being complied, here in both files that include the `main` and `squareOf` functions.

Function arguments are things that you provide to let the function do its job. Communication with functions is through these arguments and return values, if

any. For example,

Some functions take more than one parameter, like `pow`, which takes two `doubles`, the base and the exponent.

Notice that in each of these cases we have to specify not only how many parameters there are, but also what type they are. So it shouldn't surprise you that when you write a class definition, the parameter list indicates the type of each parameter. For example:

```
void printTwice (char phil) {
  cout << phil << phil << endl;
}
```

This function takes a single parameter, named `phil`, that has type `char`. Whatever that parameter is (and at this point we have no idea what it is), it gets printed twice, followed by a newline. I chose the name `phil` to suggest that the name you give a parameter is up to you, but in general you want to choose something more illustrative than `phil`.

In order to call this function, we have to provide a `char`. For example, we might have a `main` function like this:

```
int main () {
  printTwice ('a');
  return 0;
}
```

The `char` value you provide is called an **argument**, and we say that the argument is **passed** to the function. In this case the value `'a'` is passed as an argument to `printTwice` where it will get printed twice.

Alternatively, if we had a `char` variable, we could use it as an argument instead:

```
int main () {
  char argument = 'b';
  printTwice (argument);
  return 0;
}
```

Notice something very important here: the name of the variable we pass as an argument (`argument`) has nothing to do with the name of the parameter (`phil`). Let me say that again:

> **The name of the variable we pass as an argument has nothing to do with the name of the parameter.**

They can be the same or they can be different, but it is important to realize that they are not the same thing, except that they happen to have the same value (in this case the character `'b'`).

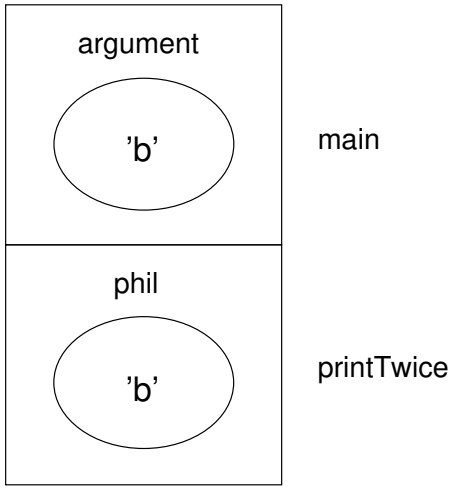
The value you provide as an argument must have the same type as the parameter of the function you call. This rule is important, but it is sometimes confusing because C++ sometimes converts arguments from one type to another automatically. For now you should learn the general rule, and we will deal with exceptions later.

3.8 Parameters and variables are local

Parameters and variables only exist inside their own functions. Within the confines of `main`, there is no such thing as `phil`. If you try to use it, the compiler will complain. Similarly, inside `printTwice` there is no such thing as `argument`.

Variables like this are said to be **local**. In order to keep track of parameters and local variables, it is useful to draw a **stack diagram**. Like state diagrams, stack diagrams show the value of each variable, but the variables are contained in larger boxes that indicate which function they belong to.

For example, the state diagram for `printTwice` looks like this:



Whenever a function is called, it creates a new **instance** of that function. Each instance of a function contains the parameters and local variables for that function. In the diagram an instance of a function is represented by a box with the name of the function on the outside and the variables and parameters inside.

In the example, `main` has one local variable, `argument`, and no parameters. `printTwice` has no local variables and one parameter, named `phil`.

3.9 Functions with multiple parameters

The syntax for declaring and invoking functions with multiple parameters is a common source of errors. First, remember that you have to declare the type of every parameter. For example

```
void printTime (int hour, int minute) {
  cout << hour;
  cout << ":";
  cout << minute;
}
```

It might be tempting to write `(int hour, minute)`, but that format is only legal for variable declarations, not for parameters.

Another common source of confusion is that you do not have to declare the types of arguments. The following is wrong!

```
int hour = 11;
int minute = 59;
printTime (int hour, int minute);   // WRONG!
```

In this case, the compiler can tell the type of `hour` and `minute` by looking at their declarations. It is unnecessary and illegal to include the type when you pass them as arguments. The correct syntax is `printTime (hour, minute)`.

3.10 Functions with results

You might have noticed by now that some of the functions we are using, like the math functions, yield results. Other functions, like `newLine`, perform an action but don't return a value. That raises some questions:

- What happens if you call a function and you don't do anything with the result (i.e. you don't assign it to a variable or use it as part of a larger expression)?
- What happens if you use a function without a result as part of an expression, like `newLine() + 7`?
- Can we write functions that yield results, or are we stuck with things like `newLine` and `printTwice`?

The answer to the third question is "yes, you can write functions that return values," and we'll do it in a couple of chapters. I will leave it up to you to answer

the other two questions by trying them out. Any time you have a question about what is legal or illegal in C++, a good way to find out is to ask the compiler.

3.11 Glossary

floating-point: A type of variable (or value) that can contain fractions as well as integers. There are a few floating-point types in C++; the one we use in this book is `double`.

initialization: A statement that declares a new variable and assigns a value to it at the same time.

function: A named sequence of statements that performs some useful function. Functions may or may not take parameters, and may or may not produce a result.

parameter: A piece of information you provide in order to call a function. Parameters are like variables in the sense that they contain values and have types.

argument: A value that you provide when you call a function. This value must have the same type as the corresponding parameter.

call: Cause a function to be executed.